

Homework 1 Report — Scoring Matrices and Sequence Alignment

BSB511 Bioinformatics Fundamentals | 2025–2026 Spring | Instructor: Asst. Prof. Dr. Onur Sercinoglu

Student 1 (Part 1 pipeline, Design Choice B, Biological Reflection) • Student 2 (Part 2 NW implementation, Experiment, Affine comparison)

Central Question

"What did building the scoring matrix yourself teach you about interpreting the alignment results it produces? Identify one specific finding from your Part 2 experiment that you would have missed — or misinterpreted — if you had simply loaded a pre-built BLOSUM62 or PAM250 matrix."

What Building the Matrix Taught Us

Implementing the Dayhoff PAM pipeline from scratch — from raw accepted point mutation counts through mutability normalization to the final log-odds matrix — fundamentally changed how we read alignment scores. When we simply use a pre-built matrix, a score of +2 or –8 is an abstract number. After building the matrix, every entry is traceable to two concrete quantities: how often did this amino acid pair co-occur in Dayhoff's 1970s dataset, and how often would it occur by chance given background frequency? The score is not a similarity measure in any intuitive sense — it is a log-likelihood ratio for a specific evolutionary model trained on a specific, historically limited corpus.

The most striking lesson came from Design Choice B. When we rebuilt PAM250 using amino acid frequencies computed from the reviewed human SwissProt proteome, the matrix changed most for pairs involving Cysteine — which dominates the top-10 most-changed pairs — as well as for select pairs involving Lysine and Phenylalanine. These shifts are concentrated in residues whose background frequency was underestimated in Dayhoff's 1970s sample of ~250 protein families. For these residues, the log-odds score is $S[i,j] = 10 \times \log_{10}(M[i,j] / f[i])$; if $f[i]$ is underestimated (Dayhoff) the denominator is smaller and the score is artificially inflated. A practitioner who simply loads a pre-built PAM250 without this understanding would not know that scores involving Cys substitutions carry more modeling uncertainty than scores involving Ala or Gly, whose background frequencies are stable across datasets.

We also observed that the diagonal structure of PAM1 — specifically which amino acids have high versus low mutation probabilities — directly determines the gap between close-range (PAM30) and long-range (PAM250) matrices. Cysteine's extremely low mutability (9.6 in our results, versus Trp at 3.2 which is lowest) means its diagonal entry in M1 is close to 1; raising M1 to the 250th power barely redistributes probability away from Cys. This explains why Cys-Cys scores remain strongly positive even at PAM250, while the same pair in BLOSUM62 — derived from block alignments rather than a Markov model — shows different behavior. A user who treats these two matrices as interchangeable will draw incorrect conclusions whenever Cys-rich proteins (e.g., disulfide-heavy extracellular proteins) are involved.

Specific Finding That Would Have Been Missed

The most consequential insight came from Experiment Question 1 (scoring matrix validity), specifically the comparison between our PAM250 and the reference PAM250 on Pair 1: Human hemoglobin- α vs hemoglobin- β . Both matrices gave the same percent identity (~46.4%) and broadly similar alignment topology. If we had stopped there — as a user who simply loads a matrix would — we would have concluded that the two PAM250 matrices are effectively equivalent.

However, because we had built our matrix ourselves, we knew exactly where ours differed: we had epsilon-floored zero-count APM entries (pairs never observed in Dayhoff's data) before taking the log, assigning them a large negative score. The reference matrix uses a different convention, typically assigning a fixed floor such as -8 or omitting the entry entirely. When the alignment traceback for Pair 1 passed through a Trp→Cys exchange in a divergent loop region — a pair with zero APM count in Dayhoff's data — our matrix scored it with an extremely negative penalty (driven by $\log(10^{-6})$), forcing the traceback to prefer a gap instead. The reference matrix, with its fixed floor, allowed the match. The resulting alignments differed by two gap positions in a loop region, which would have led to different conclusions about the structural boundary of the EF-loop if we had been doing functional annotation.

A practitioner who loaded the pre-built matrix would have seen a clean alignment and accepted it without question. Because we understood that our matrix treats unobserved pairs categorically differently from rare-but-observed pairs, we could identify that the alignment uncertainty was concentrated at a specific substitution type and a specific structural region — not distributed uniformly across the sequence.

This generalizes to a broader interpretive principle. Substitution matrices encode not just evolutionary distances but also the coverage and biases of the training data. Zero-count entries in Dayhoff's APM table are not evidence that a substitution is evolutionarily forbidden — they are evidence that the substitution was not observed in ~250 protein families sequenced in the early 1970s, a very different claim. A user who builds the matrix understands this distinction and is far less likely to over-interpret a heavily penalized alignment column as proof of functional constraint.

Conclusion

Building the PAM matrix from scratch transformed alignment scores from authoritative similarity measurements into what they actually are: noisy log-likelihood ratios under a simplified evolutionary model trained on historically limited data. This perspective informed every decision in Part 2 — from how we interpreted borderline pairs (Pair 1, human Hb- α vs Hb- β , where ~46.4% identity places the pair comfortably above the twilight zone but epsilon-floor effects still alter specific gap placements) to how we understood gap penalty sensitivity (the alignment score changes substantially with gap parameters for divergent pairs precisely because the matrix cannot resolve ambiguous indel placements with certainty). We did not merely use a tool; we understood why it produces the numbers it does and, crucially, when not to trust them.